

Warm-starting a Motion Planner using Self-Supervised Learning

1st Timo Matuszewski

*School of Computation, Information and Technology
Technical University of Munich
Munich, Germany*

2nd Gabriel Salinas

*School of Computation, Information and Technology
Technical University of Munich
Munich, Germany*

Abstract—Optimization-based motion planners produce smooth, collision-free trajectories, but whether they succeed, and how long they take, is dictated almost entirely by their initial guess. We present a warm-start network that maps a start configuration, a goal configuration and a signed distance field of the workspace directly onto the control points of a cubic B-spline trajectory. It is trained end-to-end on the same differentiable collision, joint-limit and smoothness costs the optimizer itself minimizes, so it needs no expert demonstrations and no labels of any kind. On 1000 held-out problems with irregular, non-convex obstacles, warm-starting CHOMP with the network raises the collision-free rate from 61.8% to 93.1% while reducing the mean optimizer iterations from 20.8 to 5.9 and the planning time per query from 221 ms to 67 ms. The improvement is concentrated where it matters: on the problems the network alone fails, the warm start repairs 79.8%, against 42.9% for the identical optimizer started from a straight line.

Index Terms—motion planning, trajectory optimization, self-supervised learning, warm start, B-splines, CHOMP

I. INTRODUCTION

Trajectory optimizers such as CHOMP [1], STOMP [2] and TrajOpt [3] return smooth, kinematically consistent paths at a low per-iteration cost, but the objective they minimize is strongly non-convex. They only ever find a local minimum, so the outcome is governed less by the optimizer than by where it is started. In practice the default initialization is a straight line in configuration space, which in a cluttered workspace almost always sweeps the robot body through an obstacle (Fig. 1). The optimizer must then either spend many iterations pushing the trajectory out, or it settles into a local minimum that is still in collision. On our harder benchmark, cold-started CHOMP solves only 61.8% of the queries and needs 20.8 iterations on average to decide this.

Learning the initialization is a natural remedy, and supervised warm-starting works well [4], [5]. It comes, however, with a dataset bottleneck: a large set of expert trajectories must first be produced by a classical planner, which is expensive, and the learner inherits the expert’s biases and cannot easily exceed its quality. For higher-dimensional systems this is the dominant cost of the whole approach.

Our starting point is that the objective a trajectory optimizer minimizes is already differentiable with respect to the trajectory. Rather than imitating an expert, we backpropagate that same objective directly into a network that outputs the initialization, so the label set is empty. Crucially, the network

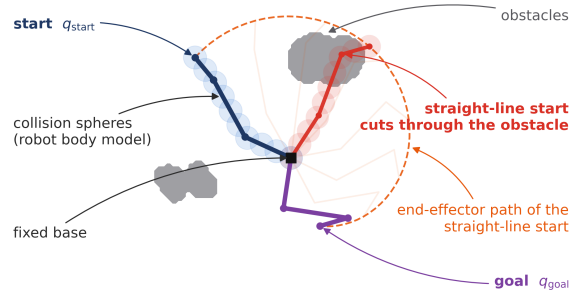


Fig. 1. The planning problem and the failure mode of a naive initialization. A 3-DOF planar arm moves from q_{start} (navy) to q_{goal} (purple) among non-convex obstacles (grey); the robot body is a chain of collision spheres. The straight line in configuration space, the standard cold start, drags the arm through an obstacle.

is not asked to solve the planning problem. It only has to land inside the basin of attraction of a feasible minimum; the classical optimizer then verifies the result and repairs it where necessary, which keeps the guarantees of the classical stage while removing most of its work.

II. RELATED WORK

Classical planners such as CHOMP [1], STOMP [2] and TrajOpt [3] formulate trajectory generation as an optimization problem balancing collision avoidance and smoothness. They produce high-quality, kinematically feasible solutions, but their performance is strongly sensitive to the initial guess and often requires many iterations in complex scenes.

To mitigate this, recent work has explored learning-based warm starts. Tenhumberg et al. [4] show that deep networks can predict structured initial guesses and accelerate convergence given a suitable environment representation, and Motion Planning Networks (MPNet) [5] directly generate feasibility-aware trajectories. Both rely on large datasets generated by classical planners, which is computationally expensive and caps performance at the quality of the expert.

Self-supervised approaches avoid that bottleneck. In inverse kinematics, self-supervised schemes [6] enforce physical and feasibility constraints through loss functions without labeled goal-state pairs, but they are limited to single configurations rather than entire trajectories.

III. PROBLEM FORMULATION

We seek an initial trajectory $\tau : [0, 1] \rightarrow \mathcal{Q}$ in the configuration space $\mathcal{Q} \subset \mathbb{R}^d$ bounded by joint limits $q_{\min}, q_{\max}$, given a start configuration q_{start} , a goal q_{goal} , and a 2D obstacle distribution represented as a Signed Distance Field (SDF) $\mathcal{S} : \mathcal{W} \rightarrow \mathbb{R}$.

A. Cubic B-Spline Trajectory Representation

Rather than predicting dense joint sequences, we parameterize $q(t)$ as a cubic B-spline over $C = 10$ control points $c = [c_1, \dots, c_C]^T \in \mathbb{R}^{C \times d}$:

$$q(t) = \sum_{i=1}^C N_{i,3}(t) c_i, \quad t \in [0, 1] \quad (1)$$

where $N_{i,3}(t)$ are the normalized cubic B-spline basis functions evaluated over a clamped uniform knot vector. Evaluating the trajectory at $T = 50$ discrete timesteps yields:

$$Q = M c \quad (2)$$

where $Q \in \mathbb{R}^{T \times d}$, and $M = A_{\text{eval}} A_{\text{interp}}^{-1} \in \mathbb{R}^{T \times C}$ is the interpolation matrix ensuring exact passage through all control points.

B. Optimization Objective

The warm-start trajectory is found by optimizing the parameter vector θ to minimize a self-supervised objective balancing safety, kinematic feasibility, and smoothness:

$$\mathcal{L}(\theta) = w_c \mathcal{L}_{\text{coll}} + w_j \mathcal{L}_{\text{lim}} + w_s \mathcal{L}_{\text{smooth}} + w_d \mathcal{L}_{\text{space}} + w_e \mathcal{L}_{\text{expl}} \quad (3)$$

The *collision* term is evaluated across the N_{sph} control spheres that model the robot body and combines Continuous Collision Detection (CCD), which expands the danger zone under high velocities, a 3-zone penetration penalty Φ , self-collision avoidance C_{self} , and exponential gradient repulsion $R(d)$. The *joint limit* term is a quadratic hinge, and *smoothness* penalizes squared joint velocities over circular differences:

$$\mathcal{L}_{\text{coll}} = \frac{1}{T} \sum_{t=1}^T \max_s \gamma_s \left(\Phi(d_{t,s}, \epsilon_{\text{eff}}) + R(d_{t,s}) + C_{\text{self}} \right) \quad (4)$$

$$\mathcal{L}_{\text{lim}} = \frac{1}{T} \sum_{t=1}^T \sum_{k=1}^d \sum_{\pm} \max(0, \pm(q_{t,k} - q_{\text{lim},k}^{\pm}))^2 \quad (5)$$

$$\mathcal{L}_{\text{smooth}} = \frac{1}{(T-1)\Delta t^2} \sum_{t=1}^{T-1} \|\text{atan2}(\sin(\Delta q_t), \cos(\Delta q_t))\|^2 \quad (6)$$

Two further terms address failure modes specific to self-supervised training. $\mathcal{L}_{\text{space}}$ penalizes the variance of the spacing between adjacent control points, preventing velocity surges through narrow passages, and $\mathcal{L}_{\text{expl}}$ forces trajectory deviation whenever the arm is in collision, which breaks the local

minimum in which the arm sits barely outside an obstacle boundary:

$$\mathcal{L}_{\text{space}} = \text{Var}(\|\text{atan2}(\sin(c_{i+1} - c_i), \cos(c_{i+1} - c_i))\|) \quad (7)$$

$$\mathcal{L}_{\text{expl}} = [\mathcal{L}_{\text{coll}}]_{\text{detach}} \cdot \max\left(0, \eta - \frac{1}{C-2} \sum_{i=2}^{C-1} \|\Delta c_i\|\right) \quad (8)$$

IV. PROPOSED METHODOLOGY

We propose a modular architecture that maps task configurations and environment representations directly to control point offsets (Fig. 2). An **environment encoder** $\mathcal{E}_{\text{env}}$, a ResNet-18 backbone modified to accept single-channel 128×128 SDF maps, compresses the obstacle geometry into $z_{\text{env}} \in \mathbb{R}^{64}$. A **state encoder** $\mathcal{E}_{\text{state}}$, an MLP, processes the task configurations augmented with forward kinematics (FK) features, $x_{\text{state}} = [q_{\text{start}}, q_{\text{goal}}, \text{FK}(q_{\text{start}}), \text{FK}(q_{\text{goal}})]$, into a motion context vector $z_{\text{state}} \in \mathbb{R}^{12}$. A **waypoint decoder** $\mathcal{D}_{\text{W}}$ then takes the concatenation $z = [z_{\text{env}}, z_{\text{state}}] \in \mathbb{R}^{76}$ and predicts offsets $\Delta c \in \mathbb{R}^{(C-2) \times d}$ for the 8 interior control points.

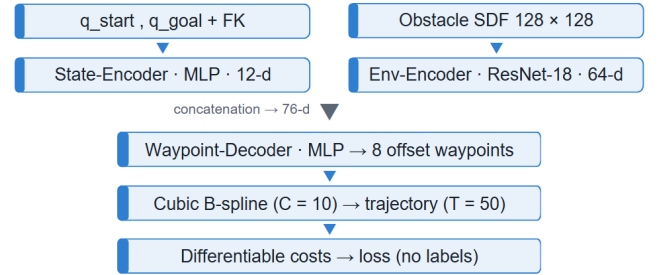


Fig. 2. Overview of the proposed neural warm-start architecture.

A. Residual Offset Formulation and Zero-Initialization

Instead of directly predicting absolute configuration waypoints, the network learns residual perturbations over a straight-line baseline c_{line} in joint space:

$$c = c_{\text{line}} + \Delta c_{\theta}(q_{\text{start}}, q_{\text{goal}}, \mathcal{S}) \quad (9)$$

To ensure training stability, the final linear layer of $\mathcal{D}_{\text{W}}$ is zero-initialized ($\Delta c_{\theta}|_{\text{init}} = 0$). Consequently, at epoch 0, the model outputs exact straight-line paths, allowing the self-supervised loss gradient to smoothly deform the trajectory away from obstacles.

V. EXPERIMENTAL SETUP

A. Robotic System and Environment Configuration

We evaluate our framework on a planar manipulator with $d = 3$ revolute joints and link lengths $l = [0.35, 0.45, 0.20]$ m, giving a maximum reach $R_{\text{max}} = 1.0$ m. Its geometry is approximated by $N_{\text{sph}} = 10$ overlapping control spheres of radius $r_{\text{sphere}} = 0.08$ m, and the joint limits are $q_{\min/\max} = \mp 0.9\pi$ rad per joint. The workspace is discretized into a 128×128 grid over a square of side $L = 2.5$ m centered at the base, and the distance grid is turned into a differentiable SDF $\mathcal{S}$ by bilinear interpolation.

Two obstacle distributions are synthesized. The *regular circles* dataset contains 1 to 3 circular obstacles with radii between 0.06 m and 0.18 m, at least 0.61 m from the base. The *voxel shapes* dataset contains 2 to 4 non-convex obstacles generated by harmonic surface perturbation and inward directional crushing, with characteristic sizes between 0.15 m and 0.30 m and a minimum base clearance of 0.35 m. A clearance margin of 0.05 m between adjacent obstacles is enforced in both cases.

B. Dataset Generation and Non-Trivial Trajectory Filtering

So that the model learns collision avoidance rather than straight-line interpolation, configuration pairs are filtered strictly. Endpoints $q_{\text{start}}, q_{\text{goal}}$ are sampled uniformly from the valid C-space and must both be collision-free with a margin of r_{sphere} against $\mathcal{S}$. The straight line between them is then evaluated at 9 intermediate points, and the pair is retained *only if* that line intersects at least one obstacle. This yields 10,000 non-trivial planning problems per distribution, split 70/15/15 into training, validation and test sets, of which 1000 are used for benchmarking. Auxiliary pre-training uses 20,000 SDF maps for $\mathcal{E}_{\text{env}}$ and 240,000 configuration pairs for $\mathcal{E}_{\text{state}}$.

C. Implementation and Training Protocol

$\mathcal{E}_{\text{env}}$ is a ResNet-18 modified for single-channel 128×128 input, $\mathcal{E}_{\text{state}}$ an MLP ($22 \rightarrow 128 \rightarrow 128 \rightarrow 12$) and $\mathcal{D}_{\mathbf{W}}$ an MLP ($76 \rightarrow 256 \rightarrow 256 \rightarrow 24$), all with ReLU activations. To reduce training complexity and stabilize representation learning, both encoders are pre-trained on auxiliary tasks and kept frozen while the decoder is trained end-to-end on $\mathcal{L}(\theta)$ from Eq. (3) with Adam, 4,000 epochs, batch size 128 and $lr = 5 \times 10^{-4}$ on a Tesla T4 GPU. The trajectory uses $\Delta t = 0.1$ s, a safety margin $\epsilon_{\text{eff}} = 0.15$ m and an exploration threshold $\eta = 0.5$, with loss weights $(w_c, w_j, w_s, w_d, w_e) = (350, 1, 1, 2, 5)$ for the circular and $(400, 1, 1, 2, 5)$ for the voxel model, which differ only in the collision term.

D. Baseline Methods and Evaluation Metrics

Each model is benchmarked on the 1000 held-out test problems of its own dataset against **Network only** (a single forward pass and the B-spline evaluation, no optimizer), **CHOMP (cold)** (the conventional baseline, initialized with the straight line in configuration space), and our **Network $\rightarrow$ CHOMP (warm)**, in which the predicted control points are lifted through the same clamped matrix M the model uses and handed to the optimizer as its initial trajectory. Two **+extra losses** variants additionally give CHOMP the waypoint-spacing and exploration terms, ablating whether the terms that stabilize *training* also help as per-problem search aids.

Optimizer. CHOMP is reimplemented on top of the same cost terms the network is trained on, with the covariant update of [1] and the endpoints pinned. One deviation mattered: the fixed covariant step overshoots badly on this stiff cost and can leave a warm-started trajectory worse than it began, so the step is halved by backtracking line search until the cost

TABLE I
BENCHMARK ON $N = 1000$ HELD-OUT PLANNING PROBLEMS PER DATASET. THE PROPOSED PIPELINE IS HIGHLIGHTED IN BOLD.

Method	Crash-free [%]	Mean iters	Time / query [ms]
<i>Circular obstacles (wsp_general_0907)</i>			
Network only	86.3	n/a	9.1
CHOMP (cold)	71.9	14.6	144.6
Network $\rightarrow$ CHOMP (warm)	97.6	3.1	35.0
<i>Voxel shapes (wsp_general_shapes_1707)</i>			
Network only	67.4	n/a	8.2
CHOMP (cold)	61.8	20.8	220.9
Network $\rightarrow$ CHOMP (warm)	93.1	5.9	66.6

decreases. The optimizer runs to convergence rather than for a fixed number of steps, under identical stopping settings for every method, which is what makes iteration counts and wall-clock times comparable, and every variant is given the loaded model’s own loss weights, so the learned and classical stages pursue an identical objective.

Metrics. Scoring is deliberately weight-independent: a trajectory counts as *crash-free* if the surfaces of all N_{sph} collision spheres stay clear of every obstacle at all T timesteps, the same criterion used to build the dataset. We also report iterations to convergence, planning time per query (CPU, batch size 1, i.e. per-problem latency rather than throughput), minimum clearance, the smoothness cost, and the number of trials violating a joint limit.

VI. RESULTS AND DISCUSSION

A. Warm Starting Improves Feasibility and Speed at Once

Table I summarizes the benchmark and Fig. 3 shows the four headline metrics on the voxel shape distribution. Warm-starting CHOMP with the network raises the crash-free rate from 61.8% to 93.1%, a gain of 31.3 percentage points, while simultaneously reducing the mean iteration count from 20.8 to 5.9 (3.5 $\times$) and the planning time per query from 221 ms to 67 ms (3.3 $\times$). On the circular distribution the picture is the same: 71.9% $\rightarrow$ 97.6%, 14.6 $\rightarrow$ 3.1 iterations, and 145 ms $\rightarrow$ 35 ms.

This is the non-obvious part. A learned initialization is usually presented as a way of buying speed, and one would expect to pay for it in solution quality; here there is no trade-off, because the network is not adding work in front of the optimizer but removing work from it. Tracking the cost over iterations confirms this: the warm start begins at a cost the cold start only reaches after roughly ten iterations, and its median converges below the level the cold start plateaus at, so the two are not descending into the same minima.

Notably, cold CHOMP ends up *below* the network alone on both distributions (71.9% vs. 86.3%, and 61.8% vs. 67.4%) while costing 16 to 27 times the computation: it converges into colliding local minima, precisely the failure mode the warm start is designed to avoid.

B. Where the Improvement Comes From

The averages hide a strongly bimodal distribution, so the mean of 5.9 iterations does not describe a typical run. Splitting the 1000 voxel problems by whether the network alone already

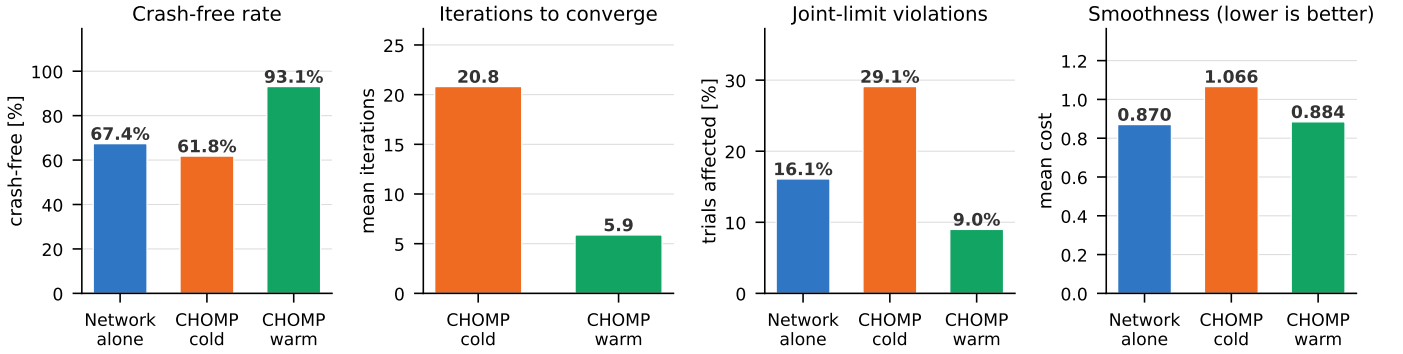


Fig. 3. The four headline metrics on the voxel shape test set ($N = 1000$). Warm starting improves all of them at once: it is the most reliable, converges in a fraction of the iterations, respects the joint limits on more trials than either baseline, and is as smooth as the network’s own output. Iteration counts are undefined for the network, which performs a single forward pass.

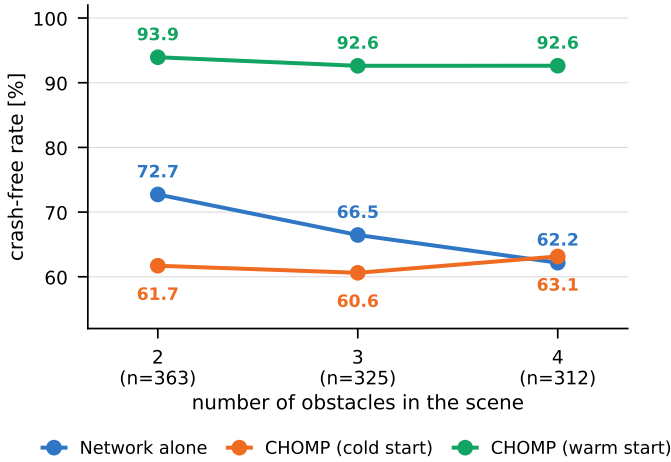


Fig. 4. Crash-free rate against scene difficulty on the voxel shape test set. The warm-started pipeline stays flat as the scene fills up, whereas the network alone degrades steadily.

solved them yields two regimes: on the 674 problems (67.4%) it solves, the optimizer only has to refine an initialization that already lies in the basin of a feasible minimum, whereas the remaining 326 demand substantially more work.

For the 326 problems the network fails, the warm start is decisive: it repairs 260 of them (79.8%), whereas cold CHOMP solves only 140 (42.9%) of the same subset. The optimizer, the cost weights and the time budget are identical; the only difference is the initialization. It is therefore the initialization, not the optimizer, that is doing the work. Three feasible trajectories regress and 66 remain infeasible, which accounts for the residual 6.9% failure rate. The circular set gives 118 of 137 repaired (86.1%) versus 80 (58.4%) cold.

C. Path Quality is Not Traded Away

Higher feasibility at lower cost would be a poor deal if it were paid for with erratic trajectories, which is not the case here (Fig. 3). Escaping a collision from a bad initialization means swinging the arm far, and cold CHOMP will leave the feasible joint range to do so: it violates joint limits on 291 of

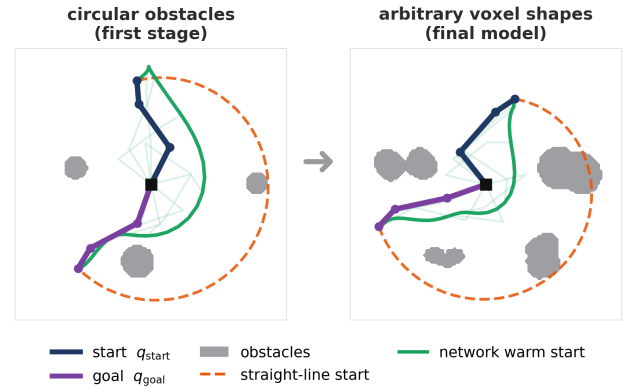


Fig. 5. The same method on both obstacle distributions. Left: circular obstacles, the first development stage. Right: arbitrary voxel shapes, the final model. In both cases the straight-line end-effector path (orange, dashed) collides while the predicted warm start (green) routes around the obstacles.

1000 voxel trials, by up to 3.00 rad, against 90 trials warm-started, and its mean minimum clearance of -0.029 m puts the average cold solution inside the safety margin, whereas the warm-started one keeps $+0.030$ m. Smoothness improves in step, and the network never learned the erratic behavior in the first place.

D. Robustness and Generalization

The pipeline was first developed on the circular obstacle distribution and then transferred to arbitrary voxel shapes by exchanging only the dataset, leaving architecture, losses and training protocol untouched (Fig. 5). The learned representation is a signed distance field rather than a parametric obstacle list, so nothing in the model is specific to circles. Performance also holds up as scenes get more crowded (Fig. 4): the warm-started crash-free rate is 93.9%, 92.6% and 92.6% for 2, 3 and 4 obstacles, essentially flat, while the network alone degrades from 72.7% to 62.2%. The optimizer absorbs the difficulty the learned stage cannot, and the combined pipeline is the only one whose reliability does not depend on how crowded the scene is.

E. Ablation and Limitations

Giving the optimizer the exploration and waypoint-spacing terms the *network* is trained with is neutral on feasibility (93.1% $\rightarrow$ 93.3%, 97.6% $\rightarrow$ 97.6%) at 1.5 to 2.2 times the wall-clock time, and it destabilizes a cold search badly (worst-case smoothness 89.7 versus 3.6). These terms earn their place during training but not as per-problem search aids.

Two caveats remain. Timings are CPU, single sample, so they report per-problem latency rather than throughput; batched GPU inference would widen the network’s advantage. And warm-started CHOMP occasionally regresses an already feasible trajectory (3 and 5 trials), because the line search guarantees monotone descent in *cost* and a cheaper trajectory can be marginally in collision.

VII. CONCLUSION AND FUTURE WORK

A trajectory optimizer can be warm-started by a neural network trained without a single labeled trajectory, by backpropagating the planner’s own differentiable cost into the network that produces its initialization. On non-convex obstacle scenes this yields 93.1% versus 61.8% crash-free trajectories at 5.9 instead of 20.8 iterations per query. The decomposition of that number is the more interesting finding: the warm start does not make the optimizer better at descending, it removes most of its work on two thirds of the queries and moves the remaining third into a basin from which it succeeds nearly twice as often as from a straight line.

Since the self-supervised signal is dimension-agnostic, the obvious next step is a higher-DOF arm in a 3-D workspace, where the payoff of removing expert data is far larger. Fine-tuning the frozen encoders should recover part of the 32.6% of queries the network alone still fails, and predicting a diverse set of warm starts instead of one would cover several homotopy classes at once.

REFERENCES

- [1] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- [2] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, “STOMP: Stochastic trajectory optimization for motion planning,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2011.
- [3] J. Schulman, J. Ho, C. Lee, I. Awwal, H. Bradlow, and P. Abbeel, “Finding locally optimal, collision-free trajectories with sequential convex optimization,” *International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [4] J. Tenhumberg, D. Burschka, and B. Büuml, “Speeding up optimization-based motion planning through deep learning,” *arXiv preprint arXiv:2311.08345*, 2023.
- [5] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, “Motion planning networks,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [6] Y. Xie *et al.*, “Efficient learning of fast inverse kinematics with collision avoidance,” *arXiv preprint arXiv:2311.05938*, 2023.